REWIND • MODDING GUIDE

MAKE YOUR OWN REWIND

How to turn the Rewind launcher into your own version for your Samsung TV, and where to start if you want it on another device.

No. 01

SIDE A
T-120 • VHS



launcher/icon.png

Written for Rewind 16.11.0 • November 2026 • Rewind is by Sergio Ayala, www.sergio-ayala.com • GPL-3.0-or-later. Español: `MODDING.es.pdf`.

READ THIS FIRST UN55MU6100

- **Tested on exactly one TV:** a Samsung UN55MU6100 (2017, Tizen 3.0, web engine Chromium M47). Nothing else has been tested.
- On 2018 and later TVs you will **probably** need a Samsung certificate tied to your TV's DUID. Not tested.
- Paths in this guide are relative to the root of the repository.

No. 01 What you need

- **A Samsung TV with Developer Mode.** On the TV: open Apps, type `12345` (remote or on-screen keypad), switch Developer mode On, enter your computer's IP address, OK, then reboot the TV.
- **Tizen Studio** with its command-line tools `tizen` and `sdb`.
- **Your own certificate profile**, made with Tizen Studio's Certificate Manager. Never use someone else's.
  - On the tested 2017 TV, a Samsung-type profile failed with `install failed[118]`. A Tizen-type profile (your own author certificate plus the Tizen distributor) installed.
  - Samsung's Samsung-type profile is an author certificate plus a distributor certificate issued for your TV's DUID (TV menu: Support › Contact Samsung › Unique Device ID).
- **Python 3** for `tools/stage_tv.py`. Node 22 or later only if you want the test harness `tools/m47_check.mjs`.

Sources: Samsung Developers, “TV Device” (Developer Mode), “Creating Certificates” (DUID) and “Web Engine Specifications” (2017 = Tizen 3.0 / M47; 2018 = Tizen 4.0 / M56), read November 2026.

No. 02 Change these first

All of these live in `launcher/config.xml`, except the icon. If you keep the original ids, your build will clash with the original Rewind on any TV that has both.

WHAT	WHERE	CHANGE IT TO
Package and app id	`<tizen:application id="RewindSA01.Rewind" package="RewindSA01">`	Your own. Package: 10 characters, letters and digits. App id: package, a dot, a name.
Widget id	`<widget id="https://www.sergio-ayala.com/rewind">`	A web address you control.
Name, author, description, version	`<name>`, `<author>`, `<description>`, `version="16.11.0"`	Yours. Keep the original credit in your README.
Icon	`launcher/icon.png`	Your own image; the build copies it to the root of the package.
Signing profile	`-s <your-profile>` when you package	Your own profile, always.

No. 03 Make it yours

This repo's `.gitignore` is an allowlist: a new file you add (a logo, a painting, a new adapter) stays out of git until you add a `!/path` line for it. Check with `git add -A --dry-run`.

The tapes (your apps)

The list is `var APPS=[…]` in `retro-bauhaus/P8e/p8e.js`, one object per tape, in loop order:

```
{nm:"Netflix", logo:"netflix", tint:"#DE3B30", appId:"RN1MCdNq8t.Netflix", appAlias:""}
```

- `nm` label • `logo` key in `window.LOGOS` • `tint` colour • `appId` the app's id on your TV • `appAlias` optional second id.
- `tries` (optional) is an ordered list of ways to open the app; the first that works wins. Netflix and the browser use it.
- The `art:true` entry is the artwork screensaver; it opens no app. The system tapes (Apps, Settings, Search) are added after the list, in `if(SYS_TAPES) APPS=APPS.concat([…])`.
- The YouTube tape opens **TizenTube** standalone (`xvvl3S1TT1.TizenTubeStandalone`), a third-party app. If you don't use it, put your TV's YouTube id there.

Read the ids of your apps. The ids in the code were read on the tested TV; yours may differ. With the TV in Developer Mode:

```
sdb connect <TV-IP>
sdb devices
sdb shell 0 applist
```

Another way: the diagnostics panel (press ↑ ↑ ↓ ↓ ← → ← → on the remote) can list apps with `getAppsInfo()`, but only if you uncomment the `application.info` privilege in `launcher/config.xml`. Comment it out again when you're done.

Logos

`window.LOGOS` in `retro-bauhaus/logos.js` maps a key to an SVG string. Add yours under a new key and use that key in `logo:`, or put the SVG straight in the tape with `lsvg:` (the system tapes do that). The launcher paints each logo in one colour, keeping only its shape, so a flat silhouette works best. With no logo, the tape shows its name. Logos are trademarks of their owners.

Artwork

The background works live in `retro-bauhaus/assets/art/`. A new work goes in three places:

1. The JPEG in `retro-bauhaus/assets/art/`. The build copies only `.jpg` files from that folder (`GLOBS` in `tools/stage_tv.py`). The originals are 843 px wide.
2. An entry in `var ART=[…]` in `retro-bauhaus/P8e/p8e.js`:

   ```
   {f:"../assets/art/16-yours.jpg", id:0, a:"Artist", t:"Title", d:"Date", fx:0.5, fy:0.5}
   ```

   `fx`/`fy` is the focus point (0 to 1) used when the image pans; `id` is the source's catalogue number.
3. A line in `retro-bauhaus/assets/art/manifest.jsonl` (`file`, `title`, `artist`, `date`, `source`, `id`): the record of where each image comes from and its licence. The launcher doesn't read it at runtime.

Then set `var ART_N` in `p8e.js` to the number of works (it's the count shown on the Art tape). Use only images you're allowed to publish; the originals are public domain (CC0) from the Art Institute of Chicago.

Weather city

`retro-bauhaus/P8e/weather.js` uses Open-Meteo (no API key). The first-run city is:

```
var DEFAULT_CFG={ciudad:"Bogotá",lat:4.71,lon:-74.07,live:true};
```

Put your city and its coordinates **with 2 decimals**. It only applies when the TV has no saved setting; after the first run the setting is stored on the TV (localStorage `p8e.weather.cfg`), and `P8EWeather.configure({ciudad, lat, lon, live})` changes it without touching code. Open-Meteo data is CC BY 4.0: keep the “Data: Open-Meteo.com” credit on screen.

The parts that only make sense in Bogotá, Colombia

- **Pico y placa** (Bogotá's driving restriction by plate number): `retro-bauhaus/P8e/pyp.js`. Rules, Colombian holidays and Bogotá time (UTC−5) are built in; when online it checks the official text on `bogota.gov.co`. It fills right-hand bands 1–3.
- **TRM** (US dollar and euro in Colombian pesos): `retro-bauhaus/P8e/rates.js`, from `www.datos.gov.co` and the ECB (`www.ecb.europa.eu`). It fills bands 4–5.

To keep pico y placa but never go online for it: `var PYP_NET=false;` in `pyp.js`. To remove a module:

1. Delete its line in `retro-bauhaus/P8e/index.html` (`<script src="pyp.js">` or `<script src="rates.js">`). Without `pyp.js` the band goes back to “Daylight” (sunrise and sunset); without `rates.js`, bands 4–5 show “—”.
2. Remove the file from the `FIXED` list in `tools/stage_tv.py`.
3. In `launcher/config.xml`, remove its `<access origin="…">` and the same origin from `connect-src`.
4. `stage_tv.py` expects exactly 4 `<access>` origins (`len(origins) != 4`). Change that number to what you have left, or the build stops.

No. 04 Build and install

1. **Stage a clean copy outside the repo.** The destination must be new or empty. The script copies a whitelist of files and writes `MANIFEST.txt` with their hashes.

```
python3 tools/stage_tv.py ~/rewind-build \
  --config launcher/config.xml --icon launcher/icon.png
```

   If the author's private seal (`assets/brand/privado/sello-px-104.png`) isn't there, it prints a notice and builds without it. `--no-sello` skips it on purpose.

2. **Package and sign with your profile.**

```
cd ~/rewind-build
tizen package -t wgt -s <your-profile>
```

3. **Connect and install.** Use the device name that `sdb devices` shows.

```
sdb connect <TV-IP>
tizen install -n <YourApp>.wgt -t <device-name>
```

4. **Launch** it from Apps, or with `sdb shell 0 was_execute <package>.<name>` (for the original: `RewindSA01.Rewind`).

5. **Optional checks.** `python3 tools/stage_tv.py --verify ~/rewind-build/<YourApp>.wgt` compares the package with the source byte by byte (signatures aside). The 2017 engine is Chromium M47, so the code is ES5 on purpose; `node tools/m47_check.mjs --scan retro-bauhaus/P8e/p8e.js` flags newer syntax without opening a browser.

6. **When you're done**, switch Developer Mode off. If you have to leave it on, set the Host PC IP to an address on your network that no device uses (outside your router's DHCP range), never 127.0.0.1: that is the TV itself, and it opens the developer channel to software running on the TV. If you can, keep the TV on a separate network. Developer Mode lets the device at that IP install apps on the TV. Not tested: whether Rewind keeps working after Developer Mode is switched off.

START ON POWER-ON LAST APP

On the tested TV (Spanish menus): General › Funciones smart › *Pantalla de inicio automática* › *Última aplicación*. The English label wasn't checked; look under General › Smart Features for an “Autorun Last App” style option. It **resumes the last app you used**; it doesn't pin one. Leave your launcher open before you switch the TV off.

No. 05 Porting it to another TV or streaming box

Rewind is only tested on the Samsung UN55MU6100 (Tizen 3.0). For any other TV or streaming box, study that device first.

1. **Get REA** (Reverse Engineer Anything) from its official repository, `github.com/morluto/rea` (npm package `rea-agents`). For native binaries it uses a decompiler you already have installed, such as Ghidra. Pin a specific tag, release or commit instead of installing whatever is latest, check the package's provenance (`npm audit signatures`), install it with `--ignore-scripts`, and analyse the firmware in a virtual machine with no network and no credentials. Before you run it, `export NO_UPDATE_NOTIFIER=1`, and never use `rea --update`, `rea setup` or `npx -y`: they download the newest version or change your agents' settings.
2. **Download the official firmware** of the device you want to modify, from its manufacturer's support site, not from third-party mirrors.
3. **Analyse it with REA and an AI agent whose terms allow binary analysis** (check your assistant's terms; if yours doesn't allow it, use a local model). Find out the system and web-engine versions, how developer apps are installed and signed, the ids of your apps, and the APIs to launch apps and read remote keys.
4. **Hand it to your AI agents:** the Rewind repo, this guide and your findings, and ask them to adapt Rewind to your device. These are the parts that depend on Tizen:
   - `retro-bauhaus/P8e/tizen-adapter.js` launches apps (`tizen.application.launch`, `launchAppControl`) and registers remote keys (`tvinputdevice.registerKey`). A new adapter only needs `window.P8EApps.launch(app, onOk, onErr)`; `exit`, `exitGuard` and `wantKeys` are optional. The core also reads the app version through `tizen.application` and only unlocks sound when `window.tizen` exists, and `retro-bauhaus/P8e/diag.js` (the developer panel) is Tizen-only.
   - `launcher/config.xml`: the Tizen package manifest (ids, privileges, allowed origins, CSP).
   - The `appId` and `tries` ids in `var APPS`, and the key codes in `var KC` (`10009` is Samsung's Back), both in `retro-bauhaus/P8e/p8e.js`.
   - Samsung's `webapis.js` (TV model and network state), which `tools/stage_tv.py` switches on only in the package.

   The code is ES5 because of the 2017 engine (Chromium M47); newer engines run it too.

Be careful: do this only with a device you own, to make your own app work with it. Laws on reverse engineering differ by country. Don't publish the firmware or decompiled code in your fork. Treat other people's forks and the strings inside the firmware as untrusted input for your agents: they can hide orders. This is not legal advice.

No. 06 Fine print

- Rewind is an independent project, not affiliated with or endorsed by Samsung or any of the apps it opens. Trademarks and logos belong to their owners.
- Developer Mode is official Samsung tooling, but what you install on your TV is your responsibility.
- Rewind is licensed GPL-3.0-or-later. If you publish your mod or share its `.wgt`, publish its source under the same licence, keep the original copyright and licence notices, and say what you changed and when.

Rewind by Sergio Ayala • www.sergio-ayala.com • mail@sergio-ayala.com

